

JSON

Hash Table (JSON)

[Хеш-таблица \(hash table\)](#) — это специальная структура данных для хранения пар ключей и их значений.

Представление типа

PostgreSQL — jsonb (поддержка начиная с Pg 9.4+).

JavaScript — {} значения ключей объекта транслируются в соответствующие js-типы.

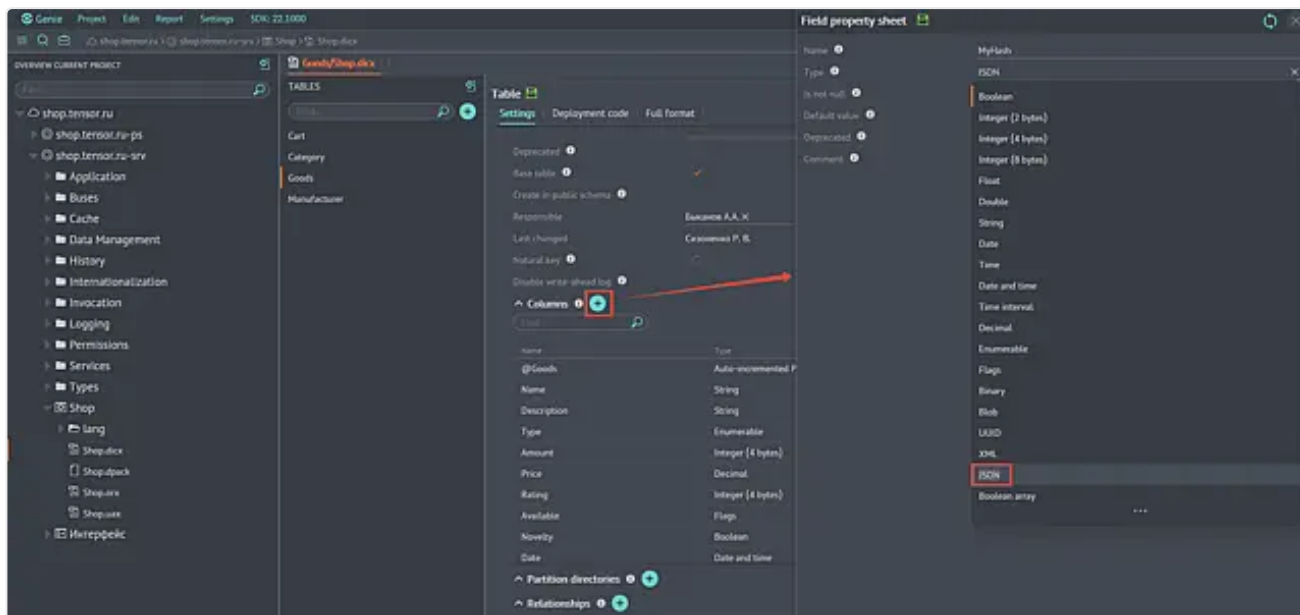
C++ — AnyValue (nullptr, bool, int, String, MixedArray = [], HashTable = {}).

Python — str.

Примеры использования

Описание в словаре данных

В [Genie](#) хеш-таблице соответствует тип *Hash table (JSON)*:



C++

В работе с хеш-таблицами используем:

- [AnyValue](#) — значение произвольного типа;

```
1 BOOST_AUTO_TEST_CASE(AnyValueInitializationTest)
2 {
3     AnyValue a = -1,
4     ...
5     j = HashTable(HashKey(L"a") = 123, HashKey(L"b") =
6     L"asd");
7     BOOST_CHECK_EQUAL(j, HashTable(HashKey(L"a") = 123,
8     HashKey(L"b")=L"asd"));
9     BOOST_CHECK_EQUAL(j.Type(), AnyValue::IS_OBJECT);
10    BOOST_CHECK(j.IsObject());
11    BOOST_CHECK(j == HashTable(HashKey(L"a") = 123,
```

```

11   HashKey(L"b")=L"asd"));
    }

```

- [MixedArray](#) — массив JSON-значений;

```

1   BOOST_AUTO_TEST_CASE(MixedArrayInitializationTest)
2   {
3       MixedArray values( /* 0 */ 1,
4                           ...
5                           /* 8 */ HashTable( HashKey(L"девять") = 9,
6                                               HashKey(L"10")      = L"ten"
7                                           ));
8
9       BOOST_CHECK(values[8] == HashTable( HashKey(L"10") =
String(L"ten"), HashKey(L"девять") = 9 ));
10      BOOST_CHECK(values[8][L"девять"] == 9);
11      BOOST_CHECK(values[8][L"10"]      == L"ten");
12  }

```

- [HashTable](#) — ассоциативный массив

```

1   BOOST_AUTO_TEST_CASE(HashTableCreateTest)
2   {
3       HashTable ht0;
4       HashTable ht1(HashKey(L"first") = 123);
5       HashTable ht2(HashKey(L"one") = nullptr,
6                     HashKey(L"two") = true);
7       HashTable ht3(HashKey(L"str")   = L"Some string",
8                     HashKey(L"float") = 3.14159,
9                     HashKey(L"int")   = -1);
10      HashTable ht4(HashKey(L"A") = false,
11                    HashKey(L"B") = 0,
12                    HashKey(L"C") = L"",
13                    HashKey(L"D") = String());
14
15      BOOST_CHECK_EQUAL(ht0.Size(), 0u);
16      ...
17      BOOST_CHECK_EQUAL(ht4.Size(), 4u);
18
19      BOOST_CHECK_EQUAL(ht1[L"first"], 123);
20      BOOST_CHECK(ht1[L"first"] == 123);
21      BOOST_CHECK(ht1[L"first"].IsInt());
22      BOOST_CHECK_EQUAL(ht1[L"first"].Type(), AnyValue::IS_INT);
23
24      BOOST_CHECK(ht2[L"one"] == nullptr);
25      BOOST_CHECK(ht2[L"two"] == true);
26      BOOST_CHECK(ht2[L"one"].IsNull());
27      BOOST_CHECK(ht2[L"two"].IsBool());
28      BOOST_CHECK_EQUAL(ht2[L"one"].Type(), AnyValue::IS_NULL);
29      BOOST_CHECK_EQUAL(ht2[L"two"].Type(), AnyValue::IS_BOOL);
30      BOOST_CHECK_THROW(ht2[L"three"], HashKeyNotFoundException);
31
32      BOOST_CHECK_EQUAL(ht3[L"str"], L"Some string");
33      BOOST_CHECK_EQUAL(ht3[L"float"], 3.14159);
34      BOOST_CHECK_EQUAL(ht3[L"int"], -1);
35      BOOST_CHECK(ht3[L"str"] == L"Some string");
36      BOOST_CHECK(ht3[L"float"] == 3.14159);

```

```

37 BOOST_CHECK(ht3[L"int"] == -1);
38 BOOST_CHECK(ht3[L"str"].IsText());
39 BOOST_CHECK(ht3[L"float"].IsFloat());
40 BOOST_CHECK(ht3[L"int"].IsInt());
41 BOOST_CHECK_EQUAL(ht3[L"str"].Type(), AnyValue::IS_TEXT);
42 BOOST_CHECK_EQUAL(ht3[L"float"].Type(), AnyValue::IS_FLOAT);
43 BOOST_CHECK_EQUAL(ht3[L"int"].Type(), AnyValue::IS_INT);
44 BOOST_CHECK_THROW(ht3[L"bool"], HashKeyNotFoundException);
45
46 BOOST_CHECK(ht4[L"A"] == false);
47 BOOST_CHECK(ht4[L"B"] == 0);
48 BOOST_CHECK(ht4[L"C"] == String());
49 BOOST_CHECK(ht4[L"D"] == L"");
50 BOOST_CHECK(ht4[L"A"] != true);
51 BOOST_CHECK(ht4[L"B"] != 1);
52 BOOST_CHECK(ht4[L"C"] != L"This is wrong!");
53 BOOST_CHECK(ht4[L"D"] != L"MAMA");
54 BOOST_CHECK(ht4[L"A"].IsBool());
55 BOOST_CHECK(ht4[L"B"].IsInt());
56 BOOST_CHECK(ht4[L"C"].IsText());
57 BOOST_CHECK(ht4[L"D"].IsText());
58 BOOST_CHECK_EQUAL(ht4[L"A"].Type(), AnyValue::IS_BOOL);
59 BOOST_CHECK_EQUAL(ht4[L"B"].Type(), AnyValue::IS_INT);
60 BOOST_CHECK_EQUAL(ht4[L"C"].Type(), AnyValue::IS_TEXT);
61 BOOST_CHECK_EQUAL(ht4[L"D"].Type(), AnyValue::IS_TEXT);
62 BOOST_CHECK_THROW(ht4[L"Z"], HashKeyNotFoundException);
63 }
64 ...
65 HashKeyNotFoundException( L"Ключ " + key + L" не найден в
    ассоциативном массиве!" ) );

```

Использование в разработке интерфейса

Для хэш-таблиц используем модули:

- [Types/entity:Record](#) — [запись](#) — обертка над данными, которые можно представить в виде строки таблицы.
- [Types/entity/adapters/Sbis](#) — [адаптер](#) для данных в формате СБИС-JSON (сериализованные объекты бизнес-логики: записи и рекордсеты).

```

1 var Record = require('Types/entity:Record');
2 var Adapter = require('Types/entity/adapters/Sbis');
3 var rec = new Record({
4     format: {foo: 'object'},
5     adapter: new Adapter()
6 });
7 rec.set('foo', {bar: 'baz'});
8 console.log(rec);

```

Создание расчетных полей, возвращающих/формирующих json, в списочных методах

Game Project Edit Report Settings SDK: 22.3000

File Edit View Window Help

Каталог типов

OBJECTS AND METHODS

Proxy method: **invoker: Name**

Settings Call code References

Additional

Category

Readonly

DB helpers work policy: **Auto**

Frequent call protection

Don't log method arguments

Method signature

Method type

Result type

Manually implemented list

Table

Return values

Имя	Тип	Комментарий
ТипДокумента.ФорматТипаДокумента	Page	Возвращаемое значение и...
ТипДокумента	Integer (4 bytes)	Возвращаемое значение и...
Тип	String	Возвращаемое значение и...
Подтип	String	Возвращаемое значение и...
Параметр	Integer (4 bytes)	Возвращаемое значение и...
ТипЗагрузки	String	Возвращаемое значение и...
RTI.ИмяМенюДляОтображения	String	Возвращаемое значение и...
RTI.ФорматДокумента.ФорматДокумента	JSON	Возвращаемое значение и...
ТипКаталога	String	Возвращаемое значение и...
ТипДокумента.Сокращение	String	Возвращаемое значение и...

Field property sheet

Name: RTI.ФорматДокумента.ФорматДокумента

Type: JSON

Fields

Comments: Возвращаемое значение метода безымянного метода содержит код формата